

Лабораторная работа № 9.

Транзакции и целостность данных

Тема: Атомарное создание заказа через Prisma interactive transaction

Цель: Реализовать создание Order и связанных OrderItem в одной транзакции Prisma, проверить rollback при ошибке и изучить ограничения целостности PostgreSQL.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте должны использоваться PostgreSQL, Prisma ORM 7 и архитектура `Route → Controller → Service`.

В базе данных должны существовать модели `User`, `Order`, `OrderItem` и `Product`, созданные в предыдущих лабораторных работах.

Таблицы, используемые в работе

Таблица	Назначение
users	Пользователи интернет-магазина
orders	Заголовки заказов
order_items	Позиции заказов
products	Каталог товаров

1. Проверьте связи Prisma

Фрагмент `prisma/schema.prisma`

```
model Order {
  id          Int          @id @default(autoincrement())
  userId      Int          @map("user_id")
  status      String       @db.VarChar(30)
  createdAt   DateTime     @default(now()) @map("created_at")
  user        User         @relation(fields: [userId], references: [id])
  items       OrderItem[]

  @@map("orders")
}

model OrderItem {
  orderId      Int          @map("order_id")
  productId    Int          @map("product_id")
  quantity     Int
  price        Int
  order        Order        @relation(fields: [orderId], references: [id])
  product      Product      @relation(fields: [productId], references: [id])

  @@id([orderId, productId])
  @@map("order_items")
}
```

Важно

Если реальные имена моделей или полей в вашем проекте отличаются, используйте фактическую схему проекта.

2. Сформируйте формат POST-запроса

```
POST /api/orders
```

```
POST /api/orders
```

```
Content-Type: application/json
```

```
{
  "userId": 1,
  "items": [
    {
      "productId": 1,
      "quantity": 2
    },
    {
      "productId": 2,
      "quantity": 1
    }
  ]
}
```

Клиент передаёт только `userId`, `productId` и `quantity`. Цена позиции заказа должна определяться сервером по `Product.price`.

Не доверяйте цене клиента

Не используйте цену товара из HTTP body как достоверную. Клиент может изменить JSON вручную.

3. Создайте Order Service

```
services/orderService.js
```

```
import { prisma } from "../config/prisma.js";
```

```
export async function createOrder(userId, items) {
  return prisma.$transaction(async (tx) => {
    const user = await tx.user.findUnique({
      where: { id: userId }
    });
  });
}
```

```
if (!user) {
  throw new Error("User not found");
}
```

```
const order = await tx.order.create({
  data: {
    userId,
    status: "new"
  }
});
```

```
for (const item of items) {
```

```

    const product = await tx.product.findUnique({
      where: { id: item.productId }
    });

    if (!product || !product.inStock) {
      throw new Error(`Product ${item.productId} is unavailable`);
    }

    await tx.orderItem.create({
      data: {
        orderId: order.id,
        productId: product.id,
        quantity: item.quantity,
        price: product.price
      }
    });
  }

  return tx.order.findUnique({
    where: { id: order.id },
    include: {
      user: true,
      items: {
        include: {
          product: true
        }
      }
    }
  });
});
}

```

4. Разберите работу транзакции

1. Prisma вызывает callback внутри `$transaction()`.
2. Все запросы к базе внутри callback выполняются через `tx`.
3. Service проверяет существование пользователя.
4. Service создаёт Order и получает его `id`.
5. Для каждой позиции Service проверяет Product.
6. Цена позиции берётся из `Product.price`.
7. После успешного завершения callback происходит COMMIT.
8. Любая ошибка внутри callback приводит к ROLLBACK.

5. Добавьте Controller

```

controllers/orderController.js

import { createOrder } from "../services/orderService.js";

export async function createOrderController(req, res, next) {
  try {
    const order = await createOrder(
      req.body.userId,
      req.body.items
    );

    res.status(201).json(order);
  } catch (error) {
    next(error);
  }
}

```

```
}  
}
```

6. Добавьте маршрут

routes/orderRoutes.js

```
import { Router } from "express";  
import { createOrderController } from "../controllers/orderController.js";  
  
const router = Router();  
  
router.post("/", createOrderController);  
  
export default router;
```

app.js

```
import orderRoutes from "../routes/orderRoutes.js";  
  
app.use("/api/orders", orderRoutes);
```

7. Проверьте успешную транзакцию

9. Выберите существующий `userId`.
10. Выберите два существующих `Product` с `inStock = true`.
11. Отправьте `POST /api/orders`.
12. Убедитесь, что API возвращает статус 201.
13. Запомните `id` созданного заказа.
14. Проверьте наличие `Order` и всех `OrderItem` в базе.

Проверка PostgreSQL

```
SELECT *  
FROM orders  
ORDER BY id DESC;  
  
SELECT *  
FROM order_items  
ORDER BY order_id DESC, product_id;
```

8. Проверьте rollback

Теперь создайте запрос, в котором первая позиция содержит существующий `Product`, а вторая — заведомо отсутствующий `productId`.

Запрос с ошибкой

```
POST /api/orders  
Content-Type: application/json  
  
{
```

```
"userId": 1,
"items": [
  {
    "productId": 1,
    "quantity": 1
  },
  {
    "productId": 999999,
    "quantity": 1
  }
]
```

Запрос должен завершиться ошибкой. Несмотря на то что Order мог быть создан до проверки второй позиции, транзакция должна отменить весь результат.

Проверка rollback

```
SELECT *
FROM orders
ORDER BY id DESC;

SELECT *
FROM order_items
ORDER BY order_id DESC, product_id;
```

Критерий атомарности

После неуспешного запроса в базе не должно появиться ни нового Order, ни частично созданных OrderItem.

9. Проверьте целостность FOREIGN KEY

Внешние ключи должны запрещать ссылки на отсутствующие родительские записи.

Пример нарушения FK

```
INSERT INTO order_items (
  order_id,
  product_id,
  quantity,
  price
)
VALUES (
  999999,
  1,
  1,
  1000
);
```

PostgreSQL должен отклонить запрос, потому что `order_id = 999999` не существует.

10. Проверьте составной PRIMARY KEY

В текущей модели пара `orderId + productId` является составным первичным ключом. Один и тот же Product нельзя добавить в один Order двумя отдельными строками.

Проверка составного ключа

```
SELECT
  order_id,
  product_id,
  quantity,
  price
FROM order_items
ORDER BY order_id, product_id;
```

11. Обсудите CHECK и validation

Если в PostgreSQL существует ограничение `quantity > 0`, база отклонит нулевое или отрицательное количество. Однако HTTP validation должна обнаруживать такую ошибку ещё до Service.

Многоуровневая защита

Validation проверяет HTTP-вход, Service проверяет бизнес-правила, а PostgreSQL гарантирует ограничения структуры и связей.

12. Самостоятельное задание

15. Добавьте проверку, что массив `items` не пуст.
16. Запретите `quantity <= 0` до выполнения транзакции.
17. Запретите создание заказа для отсутствующего User.
18. Запретите заказ Product с `inStock = false`.
19. Убедитесь, что цена `OrderItem` всегда берётся из базы.
20. Продемонстрируйте успешный COMMIT.
21. Продемонстрируйте ROLLBACK при ошибке второй позиции.
22. Объясните, какие ограничения дополнительно обеспечивает PostgreSQL.

Требования к результату

- `POST /api/orders` создаёт Order и все OrderItem.
- Все операции выполняются внутри `prisma.$transaction()`.
- Внутри транзакции используется объект `tx`.
- Service проверяет существование User и Product.
- Service проверяет `inStock`.
- Цена `OrderItem` берётся из Product.
- Успешный запрос фиксируется полностью.
- Ошибка любой позиции приводит к полному rollback.
- В базе не остаются частично созданные данные.
- Студент может объяснить роль PK, FK, UNIQUE, NOT NULL и CHECK.

Критерии оценивания

№	Критерий
1	Order Service использует interactive transaction
2	Все запросы внутри транзакции выполняются через tx

№	Критерий
3	Проверяется существование User
4	Проверяется существование Product
5	Проверяется inStock
6	Цена OrderItem берётся из Product.price
7	Order и OrderItem создаются при успешном запросе
8	POST возвращает 201
9	Ошибка второй позиции приводит к rollback
10	После rollback отсутствует частичный Order
11	После rollback отсутствуют частичные OrderItem
12	Студент понимает назначение FOREIGN KEY
13	Студент понимает составной PRIMARY KEY OrderItem
14	Validation отделена от транзакционной логики
15	Студент может объяснить COMMIT и ROLLBACK